AMARA

Where warmth is the finest luxury

Frontend Project Brief

Food Ordering Platform | Next.js + TypeScript

Setup & Struktur - API & Services - Auth & Google Login - Slicing Design

Disusun untuk dikerjakan oleh Claude Code - v1.0 - Juni 2026

Daftar Isi

1. Ringkasan Project & Konteks

Amara adalah platform pemesanan makanan (food ordering) untuk restoran fine-dining. Pelanggan memesan langsung dari meja (tanpa login wajib): melihat menu, menambahkan ke keranjang, checkout dengan nomor meja, lalu melacak status pesanan. Sisi **Admin** mengelola kategori, menu (beserta upload gambar), dan pesanan yang masuk. Backend sudah live dan terdokumentasi via Swagger.

- **Backend (live):** `https://amara-development.up.railway.app` — base path `/api`, dokumentasi di `/api-docs`.
- **Autentikasi:** JWT (email + password). Token dikirim di header `Authorization: Bearer <token>`.
- **Role:** `ADMIN` dan `USER`. Endpoint admin dilindungi (403 bila bukan admin).
- **Pengerjaan:** seluruh frontend dikerjakan oleh Claude Code, mengikuti 4 langkah: Setup → Service → Auth → Slicing.

Tujuan dokumen ini

Menjadi **single source of truth** untuk membangun frontend Amara dari nol hingga terhubung penuh ke API, dengan struktur kode yang rapi & modular (tidak campur aduk), implementasi Google Auth, dan workflow slicing design Figma yang presisi.

2. Tech Stack & Tooling

Area	Pilihan	Alasan
Framework	Next.js 15 (App Router)	SSR/CSR fleksibel, routing berbasis folder, ekosistem matang.
Bahasa	TypeScript (strict)	Type-safety penuh untuk kontrak API & komponen.
Styling	Tailwind CSS v4	Utility-first, cepat untuk slicing presisi + design tokens kustom.
HTTP Client	Axios	Interceptor request/response untuk JWT & error global.
Server State	TanStack Query (React Query)	Cache, loading/err state, refetch — rapikan data fetching.
Client State	Zustand	Cart store ringan (API tidak punya endpoint cart).
Form	React Hook Form + Zod	Validasi form login/checkout/menu yang type-safe.
Auth Google	@react-oauth/google	Google Identity Services wrapper untuk tombol & ID token.
Ikon	lucide-react	Konsisten, ringan.
Format/Lint	ESLint + Prettier	Jaga konsistensi gaya kode.

Catatan TanStack Query & Zustand bersifat opsional-direkomendasikan

Jika ingin paling minimal, service + AuthContext sudah cukup berjalan. Namun React Query sangat disarankan agar setiap halaman tidak menulis ulang logika loading/error/refetch — ini kunci agar kode 'tidak berantakan'. Cart wajib pakai state client (Zustand/Context) karena backend tidak menyimpan cart.

3. Design System (diambil dari Figma)

File Figma berisi ~45 frame: versi **Desktop (1280px)** dan **Mobile (390px)**, untuk sisi **customer** dan **admin**. Brand bernuansa fine-dining: hijau hutan dalam, sage, aksen tembaga & emas hangat.

Color Palette

Primary	Secondary	Gold	Copper	Ink
#284139	#809076	#F8D794	#B86830	#111A19

Token warna disarankan (Tailwind `theme.extend.colors`):

```
// tailwind.config.ts (potongan)
colors: {
  primary:   "#284139",  // teks/brand utama, header, tombol gelap
  secondary: "#809076",  // aksen lembut, border, state hover
  gold:      "#F8D794",  // highlight, badge, wordmark di latar gelap
  copper:    "#B86830",  // CTA utama (Reserve / Pesan), harga
  ink:       "#111A19",  // teks paling gelap / background section
  paper:     "#FBFAF6",  // background terang
}
```

Tipografi

- **Display / Heading (serif):** wordmark 'Amara' & judul hero memakai serif elegan — gunakan `Playfair Display` (atau `Cormorant`) via next/font.
- **Body / UI (sans-serif):** gunakan `Inter` untuk teks, label, tombol, tabel.
- Skala: H1 ~40-48px, H2 ~28-32px, body 14-16px, caption 12px (desktop). Mobile diturunkan proporsional.

Breakpoint & Layout

- Desain dasar dua kelas: **Mobile 390px** dan **Desktop 1280px**. Pakai pendekatan mobile-first.
- Mobile memakai **Bottom Navigation Bar**; Desktop memakai **Top App Bar**. Admin desktop memakai **Side Nav**.
- Container utama maksimum ~1200-1280px, dipusatkan, padding horizontal 16-24px.

Komponen UI dasar yang perlu dibuat (di `components/ui`)

- **Button** (variant: primary/copper, ghost, outline), **Input** (dengan ikon & state error), **PasswordInput** (toggle eye).
- **Card**, **Badge** (status pesanan berwarna), **Modal/Dialog**, **Drawer** (detail pesanan admin), **Tabs** (Masuk/Daftar).
- **Toast** (notifikasi sukses), **Skeleton** (loading), **EmptyState**, **Pagination**, **QuantityStepper**.

4. Arsitektur & Struktur Folder

Prinsip: **routing** (folder `app/`) dipisah dari **logika domain** (folder `features/`) dan **infrastruktur** (folder `lib/`). Komponen presentational murni ada di `components/`. Dengan begitu kode tidak campur aduk.

```
amara-frontend/
├─ public/                      # gambar statis, favicon, logo
├─ src/
│  ├─ app/                      # ROUTING saja (tipis, hanya rakit halaman)
│  │  ├─ (public)/              # grup rute publik (pakai layout customer)
│  │  │  ├─ page.tsx            # Landing Page
│  │  │  ├─ menu/page.tsx       # Daftar menu (list + filter + search)
│  │  │  ├─ menu/[id]/page.tsx  # Detail menu
│  │  │  ├─ cart/page.tsx       # Keranjang
│  │  │  ├─ checkout/page.tsx   # Checkout -> POST /orders
│  │  │  ├─ order/[id]/page.tsx # Lacak status pesanan (track)
│  │  │  └─ layout.tsx          # TopAppBar + Footer + BottomNav
│  │  ├─ (auth)/                # grup rute auth (layout terpusat)
│  │  │  ├─ login/page.tsx
│  │  │  ├─ register/page.tsx
│  │  │  ├─ forgot-password/page.tsx   # (UI; lihat catatan backend)
│  │  │  └─ reset-password/page.tsx    # (UI; lihat catatan backend)
│  │  ├─ admin/                 # area admin (dijaga ProtectedRoute ADMIN)
│  │  │  ├─ layout.tsx          # SideNav + guard role ADMIN
│  │  │  ├─ dashboard/page.tsx
│  │  │  ├─ menus/page.tsx      # CRUD menu + modal tambah/edit
│  │  │  └─ orders/page.tsx     # kelola pesanan + drawer detail + ubah status
│  │  ├─ layout.tsx             # ROOT: <Providers> (Auth, Query, Google)
│  │  └─ globals.css
│  ├─ components/
│  │  ├─ ui/                    # primitives: Button, Input, Card, Badge, Modal...
│  │  ├─ layout/                # TopAppBar, Footer, BottomNav, AdminSidebar
│  │  └─ shared/                # MenuCard, OrderStatusBadge, dll (reusable)
│  ├─ features/                 # MODUL DOMAIN (logic + UI per fitur)
│  │  ├─ auth/
│  │  │  ├─ components/          # LoginForm, RegisterForm, GoogleAuthButton
│  │  │  ├─ context/AuthContext.tsx
│  │  │  └─ guards/ProtectedRoute.tsx
│  │  ├─ menu/   ├─ cart/   └─ order/
│  ├─ lib/
│  │  ├─ api/
│  │  │  ├─ client.ts           # axios instance + interceptors (api.ts)
│  │  │  ├─ auth.service.ts
│  │  │  ├─ category.service.ts
│  │  │  ├─ menu.service.ts
│  │  │  └─ order.service.ts
│  │  ├─ utils/                 # formatRupiah, cn (clsx+twMerge)
│  │  └─ query.ts               # QueryClient
│  ├─ types/api.types.ts        # tipe API & DTO terpusat
│  ├─ config/env.ts             # baca env var (terpusat, type-safe)
│  └─ providers.tsx             # gabungan provider untuk root layout
├─ .env.local
├─ tailwind.config.ts
├─ tsconfig.json  (path alias @/* -> src/*)
└─ package.json
```

Aturan main agar tetap rapi

1) **app/** tidak boleh memanggil axios langsung — selalu lewat service di `lib/api` atau hook fitur.

2) Komponen di **components/ui** tidak boleh tahu soal API — murni presentational (props in, UI out).

3) Satu service = satu domain (auth, menu, category, order). Tipe respons hidup di `types/api.types.ts`.

4) Logika spesifik fitur (mis. hitung total cart) tinggal di `features/<fitur>`, bukan di halaman.

5. Langkah 1 — Setup Project & Struktur Folder

Inisialisasi project Next.js + TypeScript + Tailwind, lalu pasang dependency inti.

```
# 1. Scaffold Next.js (App Router, TS, Tailwind, ESLint, alias @/*)
npx create-next-app@latest amara-frontend \
  --typescript --tailwind --eslint --app --src-dir --import-alias "@/*"

cd amara-frontend

# 2. Dependency inti
npm install axios @tanstack/react-query zustand \
  react-hook-form zod @hookform/resolvers \
  @react-oauth/google lucide-react clsx tailwind-merge

# 3. Dev tooling
npm install -D prettier prettier-plugin-tailwindcss

# 4. Buat struktur folder kosong (lihat bagian 4)
mkdir -p src/{components/{ui,layout,shared},features/{auth/{components,context,guards},menu,cart,order},lib/{api,ut
```

Environment variables — `.env.local`

```
NEXT_PUBLIC_API_BASE_URL=https://amara-development.up.railway.app
NEXT_PUBLIC_GOOGLE_CLIENT_ID=  # isi dari Google Cloud Console (OAuth 2.0 Client ID)
```

Akses env terpusat — `src/config/env.ts`

```
export const env = {
  apiBaseUrl: process.env.NEXT_PUBLIC_API_BASE_URL ?? "https://amara-development.up.railway.app",
  googleClientId: process.env.NEXT_PUBLIC_GOOGLE_CLIENT_ID ?? "",
} as const;
```

6. Langkah 2 — api.ts (HTTP Client) + Semua Service

> **Penting — struktur token sudah diverifikasi dari Swagger**
>
> Response login Amara berbentuk **amplop**: `{ success, message, data: { token, user } }`. Jadi token ada di `response.data.data.token` dengan nama field **token** (BUKAN `accessToken`). Semua tipe mengikuti amplop ini.

6.1 Tipe API terpusat — `src/types/api.types.ts`

```
// Amplop standar semua response Amara API
export interface ApiResponse<T> { success: boolean; message: string; data: T; }

export type Role = "ADMIN" | "USER";
export type OrderStatus =
  | "PENDING" | "PAID" | "PROCESSING" | "COMPLETED" | "CANCELLED";

export interface User  { id: string; name: string; email: string; role: Role; }
export interface AuthData { token: string; user: User; }

export interface Category { id: string; name: string; createdAt?: string; }

export interface Menu {
  id: string; name: string; description?: string; price: number;
  isAvailable: boolean; imageUrl?: string;
  categoryId: string; category?: Category;
}

export interface OrderItem { id?: string; menuId: string; quantity: number; menu?: Menu; }
export interface Order {
  id: string; tableNumber: string; customerName: string; notes?: string;
  status: OrderStatus; items: OrderItem[]; totalPrice?: number; createdAt?: string;
}

// Bentuk pagination (sesuaikan dgn respons asli /menus & /orders saat dites)
export interface Paginated<T> {
  items: T[]; page: number; limit: number; total: number; totalPages: number;
}

// ---- DTO ----
export interface RegisterDto { name: string; email: string; password: string; }
export interface LoginDto    { email: string; password: string; }
export interface CreateOrderDto {
  tableNumber: string; customerName: string; notes?: string;
  items: { menuId: string; quantity: number }[];
}
export interface MenuQuery {
  page?: number; limit?: number; search?: string; categoryId?: string;
  isAvailable?: boolean; sortBy?: "name" | "price" | "createdAt";
  sortOrder?: "asc" | "desc";
}
```

6.2 HTTP Client — src/lib/api/client.ts

```
import axios from "axios";
import { env } from "@/config/env";

export const TOKEN_KEY = "amara_token";

export const api = axios.create({
  baseURL: `${env.apiBaseUrl}/api`,
  headers: { "Content-Type": "application/json" },
});

// (1) Sisipkan JWT ke setiap request otomatis
api.interceptors.request.use((config) => {
  if (typeof window !== "undefined") {
    const token = localStorage.getItem(TOKEN_KEY);
    if (token) config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

// (2) Tangani token kedaluwarsa / tidak valid secara global
api.interceptors.response.use(
  (res) => res,
  (error) => {
    if (error.response?.status === 401 && typeof window !== "undefined") {
      localStorage.removeItem(TOKEN_KEY);
      if (!window.location.pathname.startsWith("/login")) {
        window.location.href = "/login";
      }
    }
    return Promise.reject(error);
  },
);
```

6.3 Auth Service — src/lib/api/auth.service.ts

```
import { api, TOKEN_KEY } from "./client";
import type { ApiResponse, AuthData, User, LoginDto, RegisterDto } from "@/types/api.types";

export const authService = {
  async register(dto: RegisterDto) {
    const { data } = await api.post<ApiResponse<AuthData>>("/auth/register", dto);
    return data.data;
  },
  async login(dto: LoginDto) {
    const { data } = await api.post<ApiResponse<AuthData>>("/auth/login", dto);
    if (typeof window !== "undefined") localStorage.setItem(TOKEN_KEY, data.data.token);
    return data.data;                       // { token, user }
  },
  async me() {
    const { data } = await api.get<ApiResponse<User>>("/auth/me");
    return data.data;
  },
  logout() {
    if (typeof window !== "undefined") localStorage.removeItem(TOKEN_KEY);
  },
};
```

6.4 Category & Menu Service — `category.service.ts / menu.service.ts`

```
// category.service.ts
import { api } from "./client";
import type { ApiResponse, Category } from "@/types/api.types";

export const categoryService = {
  list:   async () => (await api.get<ApiResponse<Category[]>>("/categories")).data.data,
  getById:async (id: string) => (await api.get<ApiResponse<Category>>(`/categories/${id}`)).data.data,
  // Admin only
  create: async (name: string) => (await api.post<ApiResponse<Category>>("/categories", { name })).data.data,
  update: async (id: string, name: string) =>
            (await api.patch<ApiResponse<Category>>(`/categories/${id}`, { name })).data.data,
  remove: async (id: string) => { await api.delete(`/categories/${id}`); },
};

// menu.service.ts
import { api } from "./client";
import type { ApiResponse, Menu, Paginated, MenuQuery } from "@/types/api.types";

export const menuService = {
  list:    async (q: MenuQuery = {}) =>
             (await api.get<ApiResponse<Paginated<Menu>>>("/menus", { params: q })).data.data,
  getById: async (id: string) => (await api.get<ApiResponse<Menu>>(`/menus/${id}`)).data.data,
  // Admin only — multipart/form-data (name, description, price, isAvailable, categoryId, image)
  create:  async (form: FormData) =>
             (await api.post<ApiResponse<Menu>>("/menus", form,
               { headers: { "Content-Type": "multipart/form-data" } })).data.data,
  update:  async (id: string, form: FormData) =>
             (await api.patch<ApiResponse<Menu>>(`/menus/${id}`, form,
               { headers: { "Content-Type": "multipart/form-data" } })).data.data,
  remove:  async (id: string) => { await api.delete(`/menus/${id}`); },
};
```

6.5 Order Service — `order.service.ts`

```
import { api } from "./client";
import type { ApiResponse, Order, Paginated, CreateOrderDto, OrderStatus } from "@/types/api.types";

export const orderService = {
  // Public
  create: async (dto: CreateOrderDto) =>
            (await api.post<ApiResponse<Order>>("/orders", dto)).data.data,
  track:  async (id: string) =>
            (await api.get<ApiResponse<Order>>(`/orders/${id}/track`)).data.data,
  // Admin only
  list:   async (q: { page?: number; limit?: number; tableNumber?: string; status?: OrderStatus } = {}) =>
            (await api.get<ApiResponse<Paginated<Order>>>("/orders", { params: q })).data.data,
  getById:async (id: string) => (await api.get<ApiResponse<Order>>(`/orders/${id}`)).data.data,
  updateStatus: async (id: string, status: OrderStatus) =>
            (await api.patch<ApiResponse<Order>>(`/orders/${id}/status`, { status })).data.data,
};
```

6.6 Util kecil yang sering dipakai — `lib/utils`

```
// format.ts
export const formatRupiah = (v: number) =>
  new Intl.NumberFormat("id-ID",
    { style: "currency", currency: "IDR", minimumFractionDigits: 0 }).format(v);

// cn.ts — gabung className Tailwind dengan aman
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";
export const cn = (...inputs: ClassValue[]) => twMerge(clsx(inputs));
```

7. Langkah 3 — Auth Context, Protected Route & Google Auth

7.1 AuthContext + useAuth — `features/auth/context/AuthContext.tsx`

```
"use client";
import { createContext, useContext, useEffect, useState } from "react";
import { useRouter } from "next/navigation";
import { authService } from "@/lib/api/auth.service";
import type { User, LoginDto, RegisterDto } from "@/types/api.types";

interface AuthState {
  user: User | null;
  loading: boolean;
  login:    (dto: LoginDto) => Promise<void>;
  register: (dto: RegisterDto) => Promise<void>;
  logout:   () => void;
  refresh:  () => Promise<void>;
}
const AuthContext = createContext<AuthState | null>(null);

export function AuthProvider({ children }: { children: React.ReactNode }) {
  const [user, setUser] = useState<User | null>(null);
  const [loading, setLoading] = useState(true);
  const router = useRouter();

  const refresh = async () => {
    try { setUser(await authService.me()); }
    catch { setUser(null); }
  };

  useEffect(() => { refresh().finally(() => setLoading(false)); }, []);

  const login = async (dto: LoginDto) => { await authService.login(dto); await refresh(); };
  const register = async (dto: RegisterDto) => { await authService.register(dto); };
  const logout = () => { authService.logout(); setUser(null); router.push("/login"); };

  return (
    <AuthContext.Provider value={{ user, loading, login, register, logout, refresh }}>
      {children}
    </AuthContext.Provider>
  );
}

export function useAuth() {
  const ctx = useContext(AuthContext);
  if (!ctx) throw new Error("useAuth harus dipakai di dalam <AuthProvider>");
  return ctx;
}
```

7.2 ProtectedRoute (guard role) — `features/auth/guards/ProtectedRoute.tsx`

```
"use client";
import { useEffect } from "react";
import { useRouter } from "next/navigation";
import { useAuth } from "../context/AuthContext";

export function ProtectedRoute(
  { children, role }: { children: React.ReactNode; role?: "ADMIN" },
) {
  const { user, loading } = useAuth();
  const router = useRouter();

  useEffect(() => {
    if (loading) return;
    if (!user) router.replace("/login");
    else if (role && user.role !== role) router.replace("/");
  }, [user, loading, role, router]);

  if (loading || !user || (role && user.role !== role)) {
    return <div className="grid min-h-screen place-items-center">Memuat...</div>;
  }
  return <>{children}</>;
}

// Dipakai di app/admin/layout.tsx:
//   <ProtectedRoute role="ADMIN"><AdminShell>{children}</AdminShell></ProtectedRoute>
```

7.3 Implementasi Google Auth (Google Sign-In)

Status backend untuk Google Auth

Swagger Amara saat ini **HANYA** punya `POST /api/auth/register` & `POST /api/auth/login` (email+password). **Belum ada** endpoint Google. Agar Google Login berfungsi end-to-end, backend perlu menambahkan satu endpoint penukar token (frontend kirim Google ID token, backend verifikasi & balas amplop JWT yang sama).

Kontrak endpoint yang diminta ke tim backend:

```
POST /api/auth/google
Request : { "idToken": "<Google ID token (JWT) dari Google Identity Services>" }
Response: { "success": true, "message": "Login berhasil",
            "data": { "token": "<JWT Amara>", "user": { id, name, email, role } } }
# Backend memverifikasi idToken ke Google, buat/temukan user, lalu balas JWT Amara.
```

Sisi frontend (siap dipasang sekarang, aktif begitu endpoint tersedia):

```
// 1) Bungkus aplikasi dengan provider (di src/providers.tsx)
import { GoogleOAuthProvider } from "@react-oauth/google";
import { env } from "@/config/env";
// <GoogleOAuthProvider clientId={env.googleClientId}> ...children... </GoogleOAuthProvider>

// 2) Tombol — features/auth/components/GoogleAuthButton.tsx
"use client";
import { GoogleLogin } from "@react-oauth/google";
import { api, TOKEN_KEY } from "@/lib/api/client";
import { useAuth } from "../context/AuthContext";
import type { ApiResponse, AuthData } from "@/types/api.types";

export function GoogleAuthButton() {
  const { refresh } = useAuth();
  return (
    <GoogleLogin
      onSuccess={async (cred) => {
        const { data } = await api.post<ApiResponse<AuthData>>(
          "/auth/google", { idToken: cred.credential });
        localStorage.setItem(TOKEN_KEY, data.data.token);
        await refresh();
      }}
      onError={() => console.error("Google login gagal")}
    />
  );
}
```

Rencana sementara bila endpoint backend belum siap

Render tombol Google (slicing tetap sesuai design), namun beri **feature flag** (`NEXT_PUBLIC_ENABLE_GOOGLE_AUTH`). Email+password tetap jadi jalur utama yang fungsional. Saat `/api/auth/google` live, cukup nyalakan flag — tidak ada rework. Hindari memakai Firebase Auth terpisah, karena akan ada dua sistem auth dan token backend tetap dibutuhkan untuk endpoint admin.

8. Langkah 4 — Slicing Design + Connect API

Setelah fondasi (service + auth) siap, setiap halaman tinggal: **slice UI dari Figma → pasang data dari service**. Gunakan Figma MCP agar slicing presisi (warna, spacing, ukuran terbaca langsung dari node).

Workflow slicing presisi per layar (loop)

- **1. Ambil konteks node Figma** via Figma MCP `get_design_context(fileKey, nodeId)` untuk dapat ukuran, warna, teks, dan kode referensi tiap frame. Buka frame di Dev Mode untuk node-id presisi.
- **2. Bangun komponen** pakai token Tailwind (bukan hex acak). Pecah jadi sub-komponen reusable di `components/shared` (mis. MenuCard, OrderStatusBadge).
- **3. Sambungkan data** lewat service + React Query (`useQuery` untuk read, `useMutation` untuk create/update). State loading/empty/error wajib ada.
- **4. Verifikasi visual**: jalankan dev server, screenshot, bandingkan dengan frame Figma. Iterasi sampai presisi.

Contoh halaman ter-connect (Detail Menu)

```
// app/(public)/menu/[id]/page.tsx  (Server Component tipis -> Client untuk interaksi)
import { menuService } from "@/lib/api/menu.service";
import { MenuDetail } from "@/features/menu/components/MenuDetail";

export default async function MenuDetailPage({ params }: { params: { id: string } }) {
  const menu = await menuService.getById(params.id);   // bisa juga useQuery di client
  return <MenuDetail menu={menu} />;
}
```

Urutan slicing yang disarankan (dependensi)

- **Fondasi dulu:** design tokens → komponen `ui/*` → layout (TopAppBar, Footer, BottomNav, AdminSidebar).
- **Auth flow:** Login/Register (+ tab Masuk/Daftar) → Forgot/Reset (UI) → pasang AuthContext.
- **Customer core (ada API):** Landing → Menu list → Menu detail → Cart → Checkout → Order tracking.
- **Admin (ada API):** Admin Dashboard → Kelola Menu (modal + upload) → Kelola Pesanan (drawer + ubah status).
- **Sekunder/Design-only:** Profile, Order History, Payment (QRIS/Transfer), Loyalty, Reservation — lihat bagian 9.

9. Catatan Penting: Kesenjangan Backend vs Design

Design Figma lebih luas daripada API yang tersedia. Penting menyepakati ini di awal agar tidak ada halaman yang 'mati'. Berikut klasifikasinya:

Fitur di Design	Status API	Tindakan yang disarankan
Login / Register (email+password)	Tersedia	Build penuh & connect.
Google Login	Belum ada endpoint	Build UI + client flow; minta backend tambah POST /auth/google (lihat 7.3).
Lupa / Reset Password	Belum ada endpoint	Slice UI; minta endpoint forgot/reset, atau tandai 'coming soon'.
Menu list/detail, Cart, Checkout, Track	Tersedia	Build penuh & connect (cart = state client).
Order History (customer)	Parsial	GET /orders = admin only. History customer simpan orderId di localStorage lalu /orders/{id}/track.

Fitur di Design	Status API	Tindakan yang disarankan
User Profile (lihat)	Tersedia (GET /auth/me)	Tampilkan profil dari /auth/me.
Edit Profile	Belum ada endpoint	Form read-only / minta endpoint PATCH /auth/me.
Pembayaran QRIS / Transfer Bank	Tidak ada gateway	UI statis + ubah status order ke PAID (admin/simulasi). Tidak ada pembayaran nyata.
Loyalty, Reservation, Private Dining	Tidak ada endpoint	Slice sebagai UI/mock (fase lanjutan) atau tunda hingga backend siap.

Rekomendasi

Prioritaskan layar yang **didukung API** (auth, menu, cart, checkout, tracking, admin) hingga jadi produk fungsional utuh. Layar tanpa backend dibuat sebagai UI statis bertanda jelas, dan dikoordinasikan ke tim backend memakai kontrak yang sudah dirumuskan di dokumen ini.

10. API Reference (Amara API)

Base URL: `https://amara-development.up.railway.app/api` — amplop respons `{ success, message, data }`.

Method & Path	Akses	Body / Query	Keterangan
POST /auth/register	Public	{ name, email, password }	Registrasi user baru (201).
POST /auth/login	Public	{ email, password }	Login → { token, user } (200/401).
GET /auth/me	Auth	-	Profil user yang sedang login.
GET /auth/admin-only	Admin	-	Cek akses admin (403 bila bukan).
POST /categories	Admin	{ name }	Buat kategori.
GET /categories	Public	-	Semua kategori.
GET /categories/{id}	Public	-	Detail kategori.
PATCH /categories/{id}	Admin	{ name }	Ubah kategori.
DELETE /categories/{id}	Admin	-	Hapus kategori.
POST /menus	Admin	form-data: name*, price*, categoryId*, description, isAvailable, image	Tambah menu + upload gambar.
GET /menus	Public	page, limit, search, categoryId, isAvailable, sortBy, sortOrder	List + pagination + filter.
GET /menus/{id}	Public	-	Detail satu menu.
PATCH /menus/{id}	Admin	form-data (sama, semua opsional)	Update menu + ganti gambar.
DELETE /menus/{id}	Admin	-	Hapus menu.
POST /orders	Public	{ tableNumber, customerName, notes, items[] }	Buat order baru.
GET /orders	Admin	page, limit, tableNumber, status	Semua order.
GET /orders/{id}	Admin	-	Detail order.
GET /orders/{id}/track	Public	-	Lacak status order by ID.
PATCH /orders/{id}/status	Admin	{ status }	Ubah status (PENDING..CANCELLED).
GET /health	Public	-	Cek API hidup.

11. Peta Layar ☐ Route ☐ API

Layar (Figma)	Route Next.js	API yang dipakai
Landing Page	/	GET /menus (featured)
Login & Register	/login, /register	POST /auth/login, /auth/register (+/auth/google)
Lupa / Reset Password	/forgot-password, /reset-password	(belum ada — UI)
Menu List	/menu	GET /menus, GET /categories
Menu Detail	/menu/[id]	GET /menus/{id}
Shopping Cart	/cart	(state client / Zustand)
Checkout	/checkout	POST /orders

Layar (Figma)	Route Next.js	API yang dipakai
Order Progress / Track	/order/[id]	GET /orders/{id}/track
Order History	/orders (customer)	localStorage + /orders/{id}/track
User Profile	/profile	GET /auth/me
Pembayaran QRIS / Transfer	/order/[id]/pay	(UI; status -> PAID)
Admin Dashboard	/admin/dashboard	GET /orders, GET /menus (statistik)
Admin Kelola Menu	/admin/menus	GET/POST/PATCH/DELETE /menus, /categories
Admin Kelola Pesanan	/admin/orders	GET /orders, /orders/{id}, PATCH /orders/{id}/status

12. Roadmap & Milestone

Milestone	Isi	Output
M0 — Setup	Langkah 1: scaffold, env, struktur folder, tokens Tailwind, font.	Project jalan, halaman kosong ter-route.
M1 — Fondasi data	Langkah 2: client.ts + semua service + types. Uji tiap service via Swagger token.	Service teruji, type-safe.
M2 — Auth	Langkah 3: AuthContext, ProtectedRoute, Login/Register UI, Google button (flagged).	Login email/password fungsional, area admin terjaga.
M3 — UI kit + Layout	Komponen ui/*, TopAppBar, Footer, BottomNav, AdminSidebar.	Design system konsisten siap dipakai.
M4 — Customer core	Landing, Menu, Detail, Cart, Checkout, Track — connect API.	Alur pesan-makanan end-to-end jalan.
M5 — Admin	Dashboard, Kelola Menu (upload), Kelola Pesanan (ubah status).	CRUD admin lengkap.
M6 — Sekunder & polish	Profile, Order History, Payment UI, responsive QA, empty/error states.	Siap demo / rilis.

13. Strategi Model & Effort Claude (Rekomendasi)

Karena seluruh frontend dikerjakan oleh Claude Code, pemilihan model & effort yang tepat per jenis tugas akan menghemat biaya sekaligus menjaga kualitas. Prinsipnya: **Opus untuk keputusan & hal sensitif, Sonnet untuk produksi volume (slicing), thinking tinggi hanya saat benar-benar perlu.**

Fase / Tugas	Model	Effort	Alasan
Arsitektur, struktur folder, keputusan tech stack	Opus 4.8	High + Plan mode	Keputusan berdampak luas; sekali benar, hemat rework.
api.ts + service + tipe (kontrak API)	Opus 4.8	High	Presisi kontrak/tipe kritikal & jadi fondasi semua halaman.
Auth, ProtectedRoute, Google Auth	Opus 4.8	High	Keamanan & alur token; kesalahan di sini mahal.
Design tokens + komponen ui/* dasar	Opus / Sonnet	Medium-High	Fondasi visual dipakai ulang; perlu rapi sejak awal.
Slicing halaman (volume besar)	Sonnet 4.6	Medium	Berulang & terpola; cepat + hemat untuk banyak layar.
Layout responsif rumit / pixel-tricky	Opus / Sonnet	High	Butuh penalaran spasial lebih dalam.
Boilerplate, refactor kecil, util	Sonnet / Haiku	Low-Medium	Tugas ringan, kecepatan diutamakan.
Review tiap milestone (/code-review)	Opus	High / ultra	Tangkap bug & jaga konsistensi sebelum lanjut.

Taktik tambahan agar Claude Code efektif sebagai frontend engineer

- **Buat CLAUDE.md** di root (lihat Lampiran 14) berisi konvensi, struktur, & aturan main. Semua sesi membacanya → konsisten.
- **Kerjakan per milestone & commit** tiap selesai. Jangan satu sesi raksasa; potong per fitur agar konteks tetap fokus.
- **Plan mode** untuk tugas arsitektural sebelum menulis kode; setujui rencana, baru eksekusi.
- **Tutup loop visual:** selalu jalankan dev server + screenshot dan bandingkan dengan Figma (Figma MCP get_design_context) — slicing dinilai dari hasil render, bukan asumsi.
- **Paralelkan layar independen** dengan subagent saat slicing massal (mis. beberapa halaman admin sekaligus) bila diminta.
- **Satu service per domain & tipe terpusat** → kurangi konteks yang harus dibaca ulang tiap sesi.
- **Mulai tiap fitur dari kontrak data** (service + type sudah ada), baru UI — mencegah rework.

> **Ringkas: alokasi 'effort budget'**
>
> ≈20% (Opus, high) untuk fondasi: arsitektur + service + auth. ≈60% (Sonnet, medium) untuk slicing & connect halaman. ≈20% (Opus, high/ultra) untuk review milestone, layout rumit, dan polish akhir.

14. Lampiran: Isi CLAUDE.md yang Disarankan

```
# Amara Frontend — Panduan untuk Claude Code

## Stack
Next.js 15 (App Router) + TypeScript strict + Tailwind v4 + Axios + React Query + Zustand.

## Aturan struktur
- app/ = routing tipis. Dilarang panggil axios di sini; gunakan service / hook fitur.
- lib/api/*.service.ts = satu file per domain (auth, category, menu, order).
- Semua tipe & DTO ada di src/types/api.types.ts. Amplop: { success, message, data }.
- components/ui = presentational murni (tanpa API). features/<x> = logic+UI per fitur.
- Token JWT di localStorage key "amara_token", disisipkan via interceptor di lib/api/client.ts.

## Konvensi
- Harga selalu lewat formatRupiah(). className digabung pakai cn().
- Setiap fetch tampilkan state loading / empty / error.
- Warna hanya dari token Tailwind (primary/secondary/gold/copper/ink) — jangan hex acak.
- Role ADMIN dijaga oleh ProtectedRoute role="ADMIN" di app/admin/layout.tsx.

## API
Base: https://amara-development.up.railway.app/api  (dok: /api-docs)
Google Auth: tunggu POST /auth/google (lihat brief bagian 7.3). Sementara pakai feature flag.

## Definition of Done per halaman
1) Sesuai frame Figma (verifikasi via screenshot).  2) Data dari service, bukan hardcode.
3) Loading/empty/error ditangani.  4) Responsive desktop & mobile.  5) Lulus lint.
```

— Selesai. Dokumen ini dirancang agar Claude Code dapat langsung mengeksekusi Langkah 1–4 secara berurutan. —